

... / Project4-WebGPU-Forward-Plus-and-Clustered-Deferred

<> Code

Pull requests

Actions

Projects

Security

Insights



WebGPU

0 stars 53 forks 0 watching 1 Branch 0 Tags Activity

Public repository · Forked from [CIS5650-Fall-2025/Project4-WebGPU-Forward-Plus-and-Clustered-Deferred](#)

1 Branch 0 Tags Go to file t

Go to file

Add file

Code

...

This branch is [24 commits ahead of](#) CIS5650-Fall-2025/Project4-WebGPU-Forward-Plus-and-Clustered-Deferred:main .

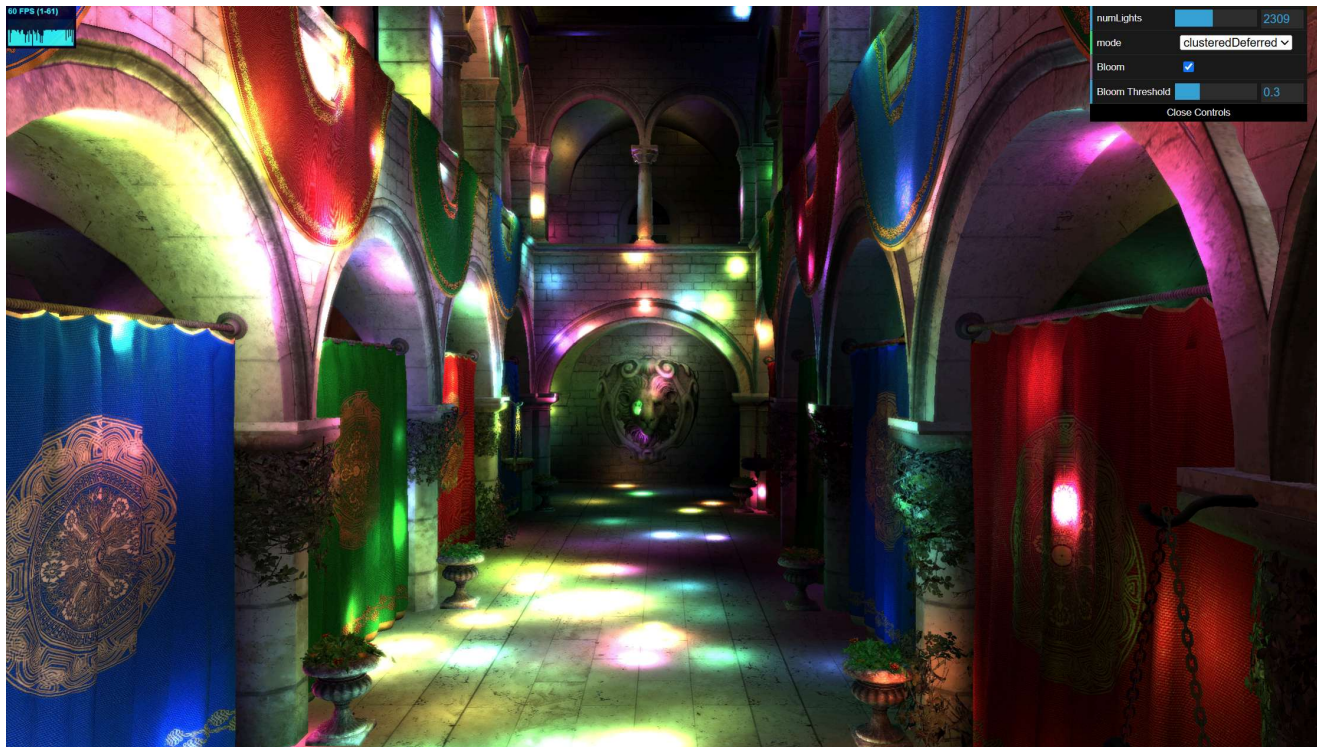
JiangmanZhao link

9a0cf19 · 1 hour ago

.github/workflows	Use Node 22 for CI	2 weeks ago
img	modify readme	1 hour ago
scenes/sponza	base code	last year
src	edge bug	6 hours ago
.gitignore	base code	last year
INSTRUCTIONS.md	update due day	2 weeks ago
README.md	link	1 hour ago
index.html	base code	last year
package-lock.json	Update package-lock.json for prev...	2 weeks ago
package.json	Update vite version	2 weeks ago
style.css	base code	last year
tsconfig.json	base code	last year
vite.config.ts	base code	last year

README

WebGL Forward+ and Clustered Deferred Shading



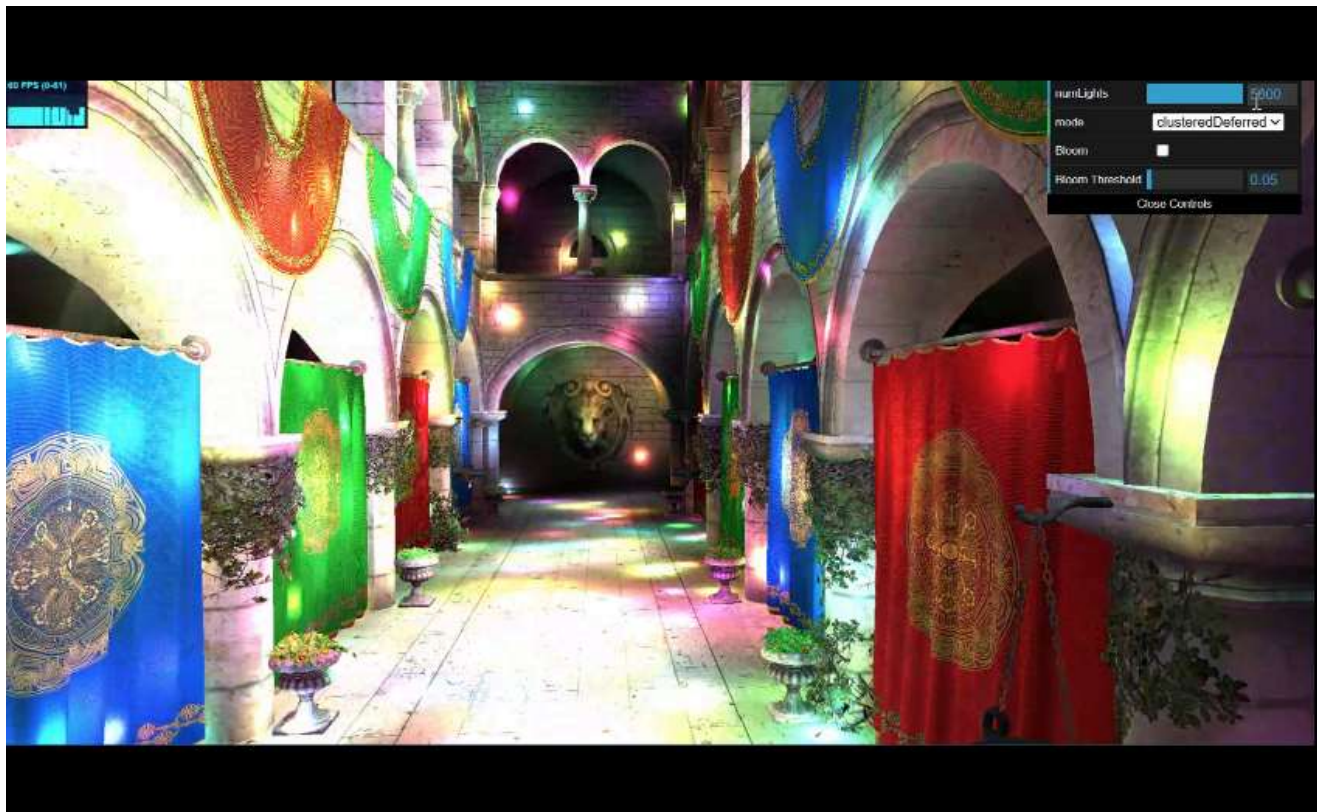
University of Pennsylvania, CIS 565: GPU Programming and Architecture, Project 4

- Jiangman Zhao
- [Lobi Zhao - LinkedIn](#), [Lobi Zhao - personal website](#).
- Tested on: **Google Chrome Version 141.0.7390.65** on Windows 11 Pro, i5-10600KF @ 4.10GHz 32GB, RTX 3080 10GB

Live Demo

- [Click Live Demo](#) 🖱️

Demo Video/GIF



Features

Naive

Baseline forward rendering with no optimizations. Tests every light against every fragment, resulting in poor performance with many lights but simple implementation.

Workflow:

- Single render pass with brute-force lighting
- Test ALL lights against EVERY fragment
- **Complexity:** $O(\text{fragments} \times \text{lights})$

Forward Plus

Optimized forward rendering using light clustering. Divides screen into tiles and assigns lights to tiles, dramatically reducing per-fragment light tests while maintaining forward rendering benefits.

Workflow:

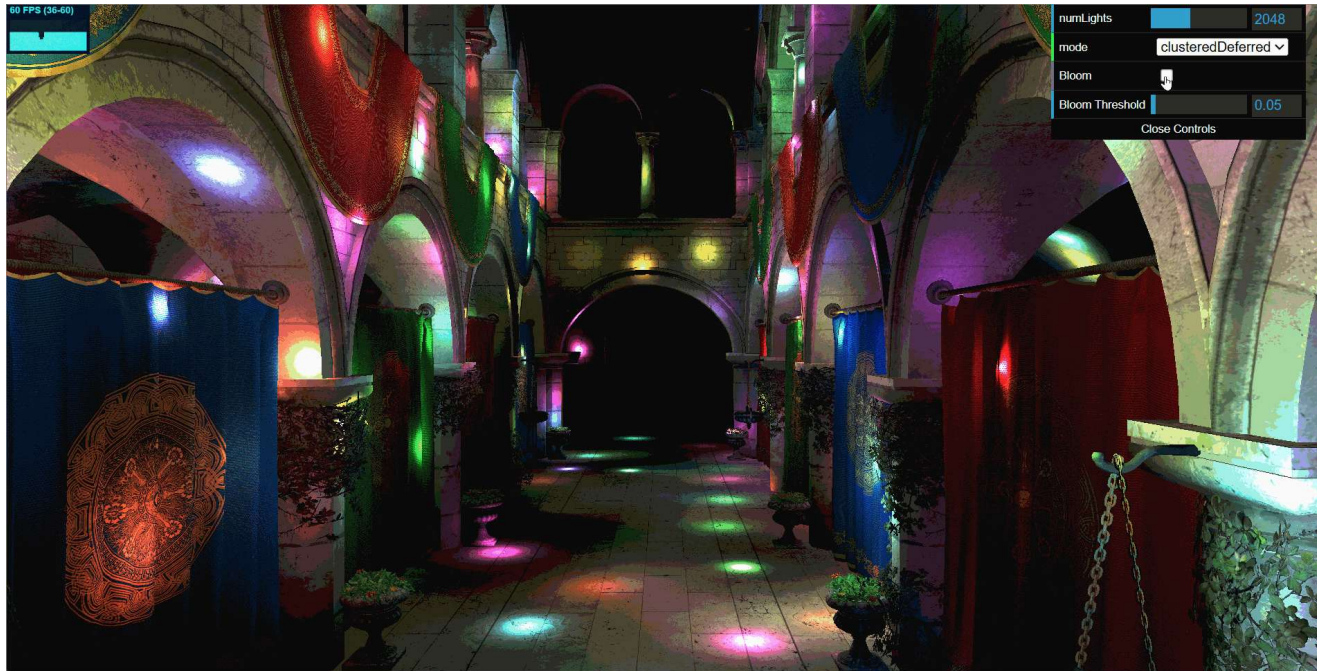
- Compute Pass: Assign lights to screen-space tiles (16×9 grid)
- Render Pass: For each fragment, determine tile and test only relevant lights

Clustered Deferred

Most efficient approach combining deferred shading with light clustering. Decouples geometry processing from lighting calculations, ideal for complex scenes with high light counts and overdraw.

Workflow:

- Compute Pass: Assign lights to screen-space tiles
- Buffer Pass: Render geometry data to textures (position, normal, albedo)
- Fullscreen Pass: Read G-Buffer, determine tile, calculate lighting only for visible pixels
- Advantage: Geometry processed once, lighting calculated only for visible pixels

Bloom

Bloom - base on Box blur

Post-processing effect that creates realistic glow around bright light sources, enhancing visual quality and realism.

Workflow:

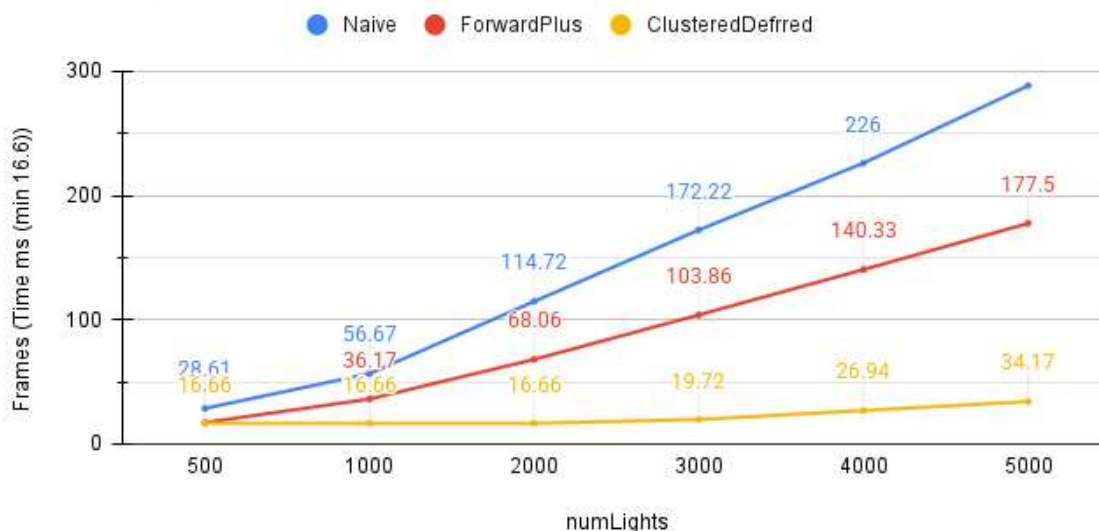
- Extract Pass: Isolate pixels brighter than threshold
- Blur Pass: Apply 11×11 box blur to bright areas
- Composite Pass: Blend blurred bloom with original scene

Implementation Files:

- `bloom_extract.fs.wgsl` - Extracts bright pixels above threshold using luminance calculation
- `bloom_blur.fs.wgsl` - Applies 11×11 box blur kernel for smooth glow effect
- `bloom_composite.fs.wgsl` - Combines original scene with blurred bloom (80% intensity)

Analysis**Performance Comparison**

Limited by display hardware, max FPS : 60



Performance Comparison: Frame Time vs Number of Lights

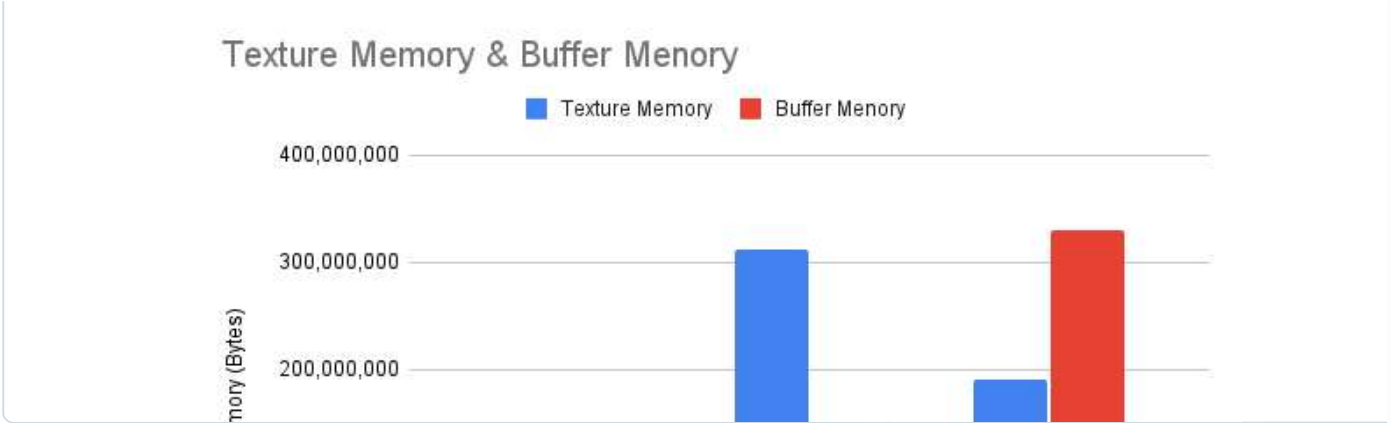
Naive Rendering: Exhibits the worst performance with linear scaling ($O(n)$). Frame time increases dramatically from 28.61ms (500 lights) to 288.33ms (5000 lights), making it unsuitable for scenes with many lights.

Forward+: Shows significant improvement over Naive through light clustering, reducing frame time by ~40% at 500 lights. However, performance still degrades linearly with light count, reaching 177.50ms at 5000 lights.

Clustered Deferred: Demonstrates superior performance and scalability. Maintains near-constant frame time (~16.66ms) up to 2000 lights due to deferred shading's ability to process only visible pixels. Even at 5000 lights (34.17ms), it outperforms Naive by 8.4× and Forward+ by 5.2×.

Clustered Deferred is the clear winner for high light count scenarios, maintaining consistent performance where other methods struggle. The G-Buffer approach eliminates redundant lighting calculations on occluded geometry, making it ideal for complex scenes with significant overdraw.

Memory Usage Analysis



Releases

No releases published

Packages

No packages published

Deployments 22

✔ github-pages 1 hour ago

[+ 21 deployments](#)

Languages

● TypeScript 80.7% ● WGSL 18.7% ● Other 0.6%